Different ways to interpret C and C++ programs

There are two modes of interpreting C and C++ programs: the interactive and the batch mode. The interactive mode resembles that of scripting languages, except that you type C and C++ program statements at the interpreter prompt, to obtain execution results immediately. In batch mode, you run one or more C and C++ source files through the interpreter to obtain your results. The interactive mode is more suited to experimentation with short code snippets; the batch mode is for when the source code is long, or organised into multiple files.

Tcc

Tcc stands for Tiny C Compiler. It is available for both 32- and 64-bit target platforms, and is heading towards full ISOC99 compliance. It is primarily a fast and efficient C compiler with a small footprint, which is able to run any C code directly without any compile/link steps. Tcc comes pre-installed as the default C compiler in Damn Small Linux. It lacks an interactive shell, but can run C code from one or multiple C source files.

To build and install Tcc, you need to have GCC installed. First download the latest Tcc source tarball from its project page (see the *References* section), and issue the following commands (replace "version" with the version number of the downloaded tarball) in a text console to extract, build and install Tcc:

```
tar -zxvf tcc-version.tar.bz2
cd tcc-version
./configure
make
sudo make install
```

If everything goes well, then running *tcc* in the console should show usage text. To test the direct C code *run* feature, save the following code as *hellotcc.c* and run it with *tcc -run hellotcc.c*:

```
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("\n Hello to FLOSS from tcc!!!\n");
    return 0;
}
```

Tcc can also run a single C source file directly as a script, if you add this *shebang* directive as the first line of the file (supply the location of the *tcc* executable as installed on your system):

```
#! <location of tcc>/tcc -run
```

You also need to mark the C source file as executable, with the following command:

```
chmod u+x  filename.c
```

Figure 1: Tcc interpreting multiple C program

Tcc can also read code from its standard input, without the need for a source file. You can pipe a line or two of C code to tcc to try this feature, as shown below:

```
echo 'main(){printf("\n"); system("uname -a"); printf("\n");}' | tcc -run -
```

To run C code organised in multiple files, add the *-run* option in, or at the end of, the list of source files. To run the stack example shown below, save it as *teststack.c*. You can issue the commands *tcc stack.c -run teststack.c* or *tcc teststack.c stack.c -run*, but Tcc gives an error if you issue *tcc -run teststack.c stack.c*.

```
#include "stack.h"

int main(int argc, char **argv)
{
    return test();
}
```

Note: The files *stack.h* and *stack.c* can be found in the zip archive of source files used in this article, available on the LFY website at *http://www.linuxforu.com/article_source_code/june10/c++codeinterpretation/source_code.zip*).

Figure 1 shows the output produced by Tcc directly running the code. Note that it indicates some warnings too, even in direct-run mode. You can appreciate the convenience and the time saved when running larger C programs in this way, and doing a final compile after development is complete.

PicoC

PicoC is a minimal interactive C code interpreter, with a very small footprint. It offers few constructs of the C language, as it's mainly targeted at small devices like embedded systems. Still, I found it useful for simple to moderate C code interpretation. The supported constructs are built in as commands, so you don't have to include standard header files while running interactive code.

You need GCC to build PicoC from source code. To build it, download the latest source tarball from its project page (see the *References* section), and run the commands given below: